SMTPRT: Performance Regression Testing and Localization for SMT Solvers Across Multiple Logics

Ziyang Wang
College of Intelligence and Computing
Tianjin University
Tianjin, China
2023244095@tju.edu.cn

Xiaohong Li
College of Intelligence and Computing
Tianjin University
Tianjin, China
xiaohongli@tju.edu.cn

Lili Quan
College of Intelligence and Computing
Tianjin University
Tianjin, China
quanlili@tju.edu.cn

Zhiping Zhou
College of Intelligence and Computing
Tianjin University
Tianjin, China
zhiping@tju.edu.cn

Yao Zhang
College of Intelligence and Computing
Tianjin University
Tianjin, China
zzyy@tju.edu.cn

Ruitao Feng
Southern Cross University
Lismore, Australia
ruitao.feng@scu.edu.au

***Abstract*—Satisfiability Modulo Theories (SMT) solvers are foundational in applications such as software verification and automated bug detection, where both correctness and performance are critical to the reliability and scalability of these systems. While existing methods predominantly focus on functional testing, performance testing has received insufficient attention, particularly regarding performance regression caused by both intentional and unintentional factors during software evolution. Current performance regression testing approaches are primarily designed for string solvers, neglecting the full spectrum of SMT theories. Furthermore, these methods often rely on time comparisons or log analysis, which makes the identification of the responsible commit slow and inefficient.**

To address the above issues, we propose a novel general-purpose testing framework, SMTPRT, that efficiently detects and localizes performance regression issues across diverse SMT solver theories. We utilize large language models (LLMs) based on genetic algorithms (GAs) to guide the search for performance regression-inducing cases. We introduce an optimized localization technique that filters irrelevant commits using code coverage, followed by a bisecting algorithm to rapidly pinpoint the responsible commit.

To thoroughly evaluate SMTPRT, we conducted extensive experiments involving six types of logic, demonstrating its superior performance. Specifically, SMTPRT successfully detected 59 regression cases, performing 3.44 times better than the baseline, and located the issues 1.16 times faster than the baseline.

***Index Terms*—SMT solver, performance regression, LLM**

I. Introduction

The Satisfiability Modulo Theories (SMT) problem extends classical satisfiability by determining the satisfiability of first-order formulas with respect to a combination of background theories. Over the past decade, SMT solvers such as Z3 [8] and CVC5 [1] have been developed to support a wide range of theories, including Boolean logic, linear and non-linear arithmetic, and string manipulation. These solvers have become foundational tools in various domains, including software and hardware verification [3], [25], [28], automated bug detection [20], type inference [14], [27], and even synthetic biology [29]. However, the correctness and performance of SMT solvers are critical to the reliability of these applications. Bugs in SMT solvers can lead to incorrect results in program verification or the omission of key test cases during test case generation. Furthermore, performance issues, particularly when handling complex path conditions, can limit the scalability and efficiency of symbolic execution, thus affecting the performance of applications that rely on SMT solvers.

While functional testing of SMT solvers has been well-explored [4]–[6], [12], [13], [15], [19], [24], [26], [31], addressing issues such as incorrect results, the topic of performance regression testing remains largely underexplored. Functional testing typically identifies bugs with clear symptoms, such as producing an incorrect "UNSAT" result for satisfiable input formulas or vice versa. Existing studies like YinYang [24], STORM [15], StringFuzz [4], and BanditFuzz [19] have successfully identified bugs in the arithmetic and string solvers of Z3 and CVC5. However, performance testing, which is vital to maintaining the scalability and efficiency of solvers, has not received the same attention. For instance, StringFuzz focuses on performance testing of string solvers, while BanditFuzz uses reinforcement learning (RL) to highlight performance gaps between solvers, and SPRFinder [31] addresses performance regressions (PRs)—unintended slowdowns introduced during the evolution of solvers. Despite these efforts, performance regressions across different theories and solver configurations remain a significant challenge.

Unlike functional bugs, performance regressions [9], [22]

Yao Zhang, Ruitao Feng are corresponding authors.

are subtle and depend on changes in solver versions, making them difficult to detect. These regressions can manifest as minor performance degradation or scalability issues, which typically do not lead to incorrect results immediately. As SMT solvers evolve, early detection of these regressions is crucial to maintaining solver efficiency and application reliability. However, existing methods mainly focus on string solvers and are difficult to adapt to other theories, where syntax and semantics can differ significantly. For example, SMTFinder primarily targets string logic types (i.e., QF_S and QF_SLIA) for testing, and the mutation generation process requires conversion to an abstract syntax tree (AST), which adds complexity to generation and debugging. Thus, handling multiple logic types and constructing ASTs is both labor-intensive and challenging. Additionally, performance regression detection typically relies on time comparisons or log analysis, which often fail to quickly pinpoint the root cause.

To fill this gap, we propose SMTPRT, a novel general-purpose framework designed to detect and localize performance regressions in SMT solvers across diverse logic types (e.g., QF_S, QF_BV). Specifically, SMTPRT first uses genetic algorithms (GAs) as a global search mechanism to expedite the identification of diverse solver performance regressions(SPR)-inducing cases. The fitness function is specifically designed to maximize runtime differences and code coverage across different SMT solver versions, ensuring that the search focuses on the most relevant code changes. GAs are well-suited for this task due to their ability to efficiently explore large search spaces, evolving a population of mutations to uncover diverse performance issues. To ensure the generality and expressiveness of mutation generation across multiple logic types, we leverage large language models (LLMs) to guide the mutation process. LLMs can generate grammatically correct and semantically meaningful SMT formula mutations for various theories, thereby enabling SMTPRT to seamlessly adapt to diverse SMT logic types without the need for manual AST construction or handcrafted rules.

Additionally, we introduce an optimized approach for identifying the responsible commit: first, we leverage code coverage information from the PR-inducing case to filter out irrelevant commits, thereby reducing the search space. Subsequently, we apply a bisecting algorithm to efficiently narrow down the specific commit that introduced the performance regression. This two-step process significantly accelerates the identification of the root cause, addressing the limitations of existing methods.

We evaluated the effectiveness of SMTPRT on the latest versions of Z3 (z3-4.13.4) and CVC5 (cvc5-1.2.1), covering six SMT logic types, including QF_S, QF_BV, QF_UF, QF_NRA, QF_FP, and QF_NIA. SMTPRT demonstrated the capability to effectively test a wide range of theoretical types, including string theory, bit-vector theory, uninterpreted function theory, floating-point arithmetic theory, nonlinear integer arithmetic theory, and nonlinear real arithmetic theory.

To ensure a fair comparison for PR detection, we selected the state-of-the-art SPRFinder as our experimental baseline. The results show that SPRFinder detected 9 regression errors with an average localization time of 0.46 hours. However, SPRFinder is primarily limited to string solvers and cannot effectively detect performance regressions in solvers using other theories. SMTPRT showed a 3.4-fold improvement in PR detection and an 16.7% faster performance in locating responsible commits, and it is applicable to multiple SMT theories. These findings highlight the advantages of SMTPRT in improving the efficiency and reliability of SMT solver performance regression detection, particularly in non-string logic contexts.

The main contributions of this paper are as follows:

- We propose a **generalized PR testing methodology**, which leverages a genetic algorithm to efficiently detect performance regression issues across multiple theories of SMT solvers through LLM-based mutations.
- We introduce a **fast and efficient localization method**, which filters out irrelevant commits using code coverage information and then applies binary search to narrow down the version range, facilitating rapid identification of performance regressions.
- We conduct **extensive experimental evaluations** to demonstrate the superior performance and effectiveness of SMTSPR compared to existing methods.
- We **open-source our implementation**, providing the community with a practical tool for performance regression testing and localization in SMT solvers. The implementation is available at the following anonymous repository: https://anonymous.4open.science/r/SMTPRT-8688/.

II. BACKGROUND

A. SMT theory

First, we explored the core technological role of SMT in formal verification. Based on the semantic specification of the SMT-LIB 2.7 [2] standard, we conducted a theoretical deconstruction of its logical classification system. This system includes complex, multidimensional relationships that meet the diverse needs of industrial verification. The standard defines 84 logical types through a three-tier structure (Basic Theory Layer, Combinatory Theory Layer, Quantifier Expression Layer), building a theoretical framework for formal verification.

The Basic Theory Layer defines seven core logical types: Linear Integer Arithmetic (QF_LIA) for program resource verification, Nonlinear Real Arithmetic (QF_NRA) for control system equations, Bit-Vectors (QF_BV) for hardware verification, Array Theory (QF_AX) for memory analysis, String Theory (QF_S) for detecting code injection, Floating-Point Arithmetic (QF_FP) for numerical precision, and Uninterpreted Functions (UF) for software equivalence verification.

The Combinatory Theory Layer supports more complex scenario verifications, such as QF_AUFBV for hardware timing verification and QF_SLIA for protocol data parsing, by logically combining types.

The Quantifier Expression Layer distinguishes between quantifier-free logic (denoted by the QF_prefix, suitable for

automated verification) and quantified logic (which has higher computational complexity). These logical types provide standardized verification benchmarks for critical fields such as chip design [18], autonomous driving, and blockchain smart contracts, ensuring the reliability of SMT solvers in various applications.

B. SMT Solvers

SMT solvers are automated reasoning tools based on Satisfiability Modulo Theories (SMT). Common solvers include Z3 [8], CVC5 [1], Yices [10], Bitwuzla [16], and MathSAT [6]. Developed by Microsoft, Z3 excels in software verification and program analysis due to its mature architecture, especially in handling linear arithmetic and bit-vectors. However, its efficiency fluctuates when dealing with nonlinear arithmetic. In contrast, CVC5 integrates machine learning and dynamic scheduling algorithms, making it proficient in solving more complex theories such as floating-point arithmetic and quantified formulas. However, its industrial application ecosystem is still under development, with further optimization needed for cross-platform deployment and exception handling.

Overall, SMT solvers play a crucial role in formal verification and automated reasoning. Each solver has distinct strengths and limitations in various scenarios. The choice of the appropriate solver depends on application needs, theoretical support, and performance requirements. Future research will continue to enhance their efficiency and scalability.

III. METHODOLOGY

A. Overview

The SMTPRT method architecture proposed in this paper is shown in Fig. 1, which includes the following two core components: **a) Performance regression testing module**, used to generate SMT test cases that trigger performance regression errors; **b) Regression tracing module**, which quickly locates the code commits that cause performance degradation.

In the performance regression testing module, to be discussed in Section III-B, we designed a dynamic generation framework based on a genetic algorithm for test case generation. This framework incorporates a mutation mechanism driven by large language models to achieve iterative optimization of test cases.

In the regression tracing module, as discussed in Section III-C, we built a commit-level regression case localization model that effectively narrows down the investigation scope by filtering relevant commits, enabling rapid identification of performance bottlenecks.

B. Regression Testing

To better generate SMT regression test cases covering multiple logical types, we adopt a genetic algorithm framework, which includes population construction, fitness function design, tournament selection, LLM-driven mutation, and population maintenance. Traditional methods require manually constructing abstract syntax trees for each logical type, whereas LLM directly generates SMT expressions for various logical types through semantic understanding, eliminating the need for manual design. The genetic algorithm automatically explores multi-logic boundaries through population diversity maintenance and adaptive optimization mechanisms. At the same time, the LLM-driven mutation and crossover selection collaboratively optimize the generation process dynamically, balancing both logical coverage and generation efficiency.

Algorithm 1 demonstrates how SMTPRT performs performance regression testing using a genetic algorithm. To quickly locate regression introduction points in subsequent work, we simultaneously input three solver versions—the latest along with two older ones (v_0, v_1, v_2) —in a single experiment run and return a set of test cases that trigger performance regression errors. Due to the special nature of SMT2 files, the logical relationships of each formula are highly intricate, making it difficult to perform splitting or crossover operations. Additionally, large language models face challenges when performing crossover operations, as they struggle to effectively recognize and handle the complex logical structures involved in the crossover process. Therefore, the algorithm omits the crossover process and retains only the mutation and selection processes.

Algorithm 1 Performance Regression Testing based on Genetic Algorithm

Input: v_0, v_1, v_2: three versions of an SMT solver
Output: R: a set of performance regression cases
Const:
α: the threshold for performance regression
β: the number of test cases in the population
c: the configuration of the generator
$max_iterations$: the maximum number of iterations

1: $R \leftarrow \emptyset$
2: $T \leftarrow$ Generator(β, c)
3: $iteration \leftarrow 0$
4: **while** $iteration < max_iterations$ **do**
5: **for** each t in T **do**
6: **if** $(tv_2 - tv_0 > \alpha)$ or $(tv_1 - tv_0 > \alpha)$ **then**
7: $R \leftarrow R \cup \{t\}$
8: $T \leftarrow T \backslash \{t\}$
9: AddToPopulation(T, t_seed)
10: **continue**
11: **end if**
12: **end for**
13: **for** each t in T **do**
14: $t' \leftarrow$ Mutate(t, action)
15: $fitness_t \leftarrow$ fitness(t_time, $t_coverage$)
16: **end for**
17: Tournament_Selection($fitness_t$)
18: $T \leftarrow Next_T$
19: $iteration \leftarrow iteration + 1$
20: **end while**
21: **return** R

First, the algorithm generates an initial set of seed test cases using the Generator function (Line 2), randomly selecting test

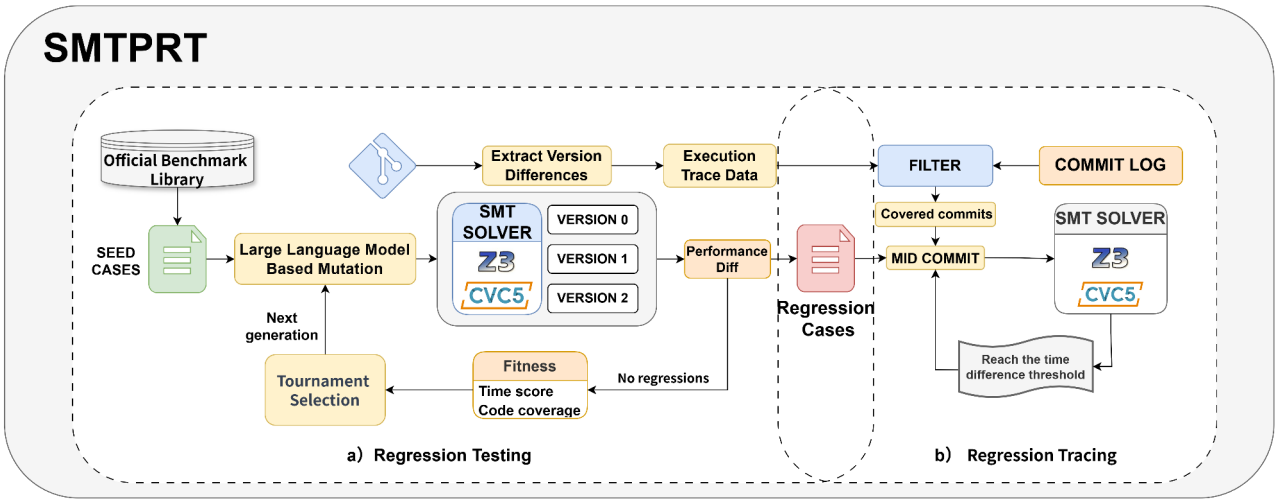


Fig. 1: Overview of SMTPRT

cases from the SMT-LIB benchmark library that match the target logical types. Then, the algorithm enters an iteration loop (Lines 3–4), running multiple versions of the SMT solver in each iteration to detect performance regression. If performance regression is triggered (Line 6), the test case is added to the regression case set R(Line 7) and removed from the test case set T, with new test cases added from the benchmark library.

If no regression errors are detected, the algorithm proceeds to the mutation and selection phase (Line 13), generating mutants and evaluating their fitness (Lines 14–15). Through tournament selection (Line 17), the best mutants are added to the test case set, updating the population and proceeding to the next iteration.

Finally, the algorithm returns all test cases in R that triggered regression errors (Line 21), completing the regression testing optimization process.

1) Population Construction: In the population construction phase, we selected standard test files from the SMT-LIB benchmark library based on the target logical types (such as QF_S, QF_BV, etc.). Considering the context length limitations of large language models, we set a file size limit of 5 KB and filtered out excessively long formulas. In the population initialization phase, we constructed the initial population based on these filtered seed files, ensuring that the test cases adhered to the syntax and semantic constraints of the target logical types while covering a diverse range of application scenarios, providing high-quality data for subsequent genetic algorithm operations.

2) Fitness Function Design: In genetic algorithms, the design of the fitness function determines the quality of individuals in the algorithm, which in turn affects the selection of the optimal solution. In performance regression detection, the fitness function not only needs to focus on the extent of performance changes but also ensure that the test cases adequately cover the modified code. Therefore, in our approach, the fitness function design combines two core metrics: time difference and coverage. The time difference reflects the performance changes between three versions when executing the same test files, while coverage measures the extent to which the modified code in the new version is tested. By considering both metrics comprehensively, the fitness function can evaluate the performance and testing coverage of the variants, thereby providing precise guidance for performance regression optimization.

Finally, the fitness function is calculated as follows:

$$\text{fitness} = \omega_1 |t_{\text{v0}} - t_{\text{v1}}| + \omega_2 \cdot \max\left(\text{cov}_{\text{v0}}, \text{cov}_{\text{v1}}\right) \quad (1)$$

In this, t_{v0} and t_{v1} represent the execution times of test cases in the old and new versions, respectively. cov_{v0} and cov_{v1} quantify the verification capability of test cases on code changes by focusing on the coverage variation in the modified code areas. ω_1 and ω_2 are tunable weights for time deviation and coverage significance, respectively.This weighted formulation establishes a multi-criteria evaluation framework, enabling the genetic algorithm to prioritize variants that balance performance efficiency (Δt) and regression impact (cov_{vn}). Adjusting ω_1 and ω_2 allows dynamic optimization for scenarios requiring either latency-sensitive tuning (e.g., increasing ω_1) or reliability-focused validation (e.g., emphasizing ω_2). The design ensures systematic convergence toward Pareto-optimal solutions, enhancing regression testing efficacy.

3) Tournament Selection: In the selection stage of the genetic algorithm, we adopted the tournament selection method. First, a small ”tournament” group is randomly selected from the current population. Then, the individuals in the group are compared based on their fitness values, and the individual with the best fitness is selected as a parent. This process is repeated until the required number of parents is selected. A significant

advantage of tournament selection is its efficiency, as it does not require sorting the fitness of the entire population.

4) LLM based Mutation: In the mutation process of the genetic algorithm, we use a large language model to mutate the selected SMT2 files. Leveraging the powerful generation capabilities of the LLM, we can diversify the original seed files and generate new formulas that meet the requirements of different logical types. Compared to the traditional AST-based methods, this approach avoids structural constraints, offering greater flexibility and adaptability. It allows for the generation of richer and more diverse SMT formulas, covering a broader range of test scenarios and effectively addressing the combination of complex logical types.

TABLE 1: Generation Prompts for SMT Formula Testing

System Prompt:
You are an expert in generating high-quality test cases that comply with the SMT-LIB standard. Please generate SMT formulas based on the user's requirements and ensure they can handle common edge cases and potential issues.
User Prompt:
The original file is {original_file}. Please generate test cases that comply with the QF_theory standard based on the following mutations:
1. Mutation Operators: Semantic-preserving mutation; Constraint-enhancing mutation; Mutation Operators; etc.
2. Allowed functions: {...}
3. Special Requirements: You may refer to issues discussed on GitHub and relevant literature.
4. Each mutation should modify no more than 5 lines and control the total number of lines after mutation.
5. Perform SMT format validation and consistency checks for the constraints.

As shown in Table 1, to better leverage large language models in generating SMT formulas that meet the required standards, we designed generation prompts to guide the model in producing high-quality test cases. These prompts are based not only on the specific requirements of the SMT-LIB standard but also incorporate relevant literature from related fields, documentation from standard libraries, and common issues and solutions discussed in GitHub community discussions. By referring to official SMT-related libraries and issues on GitHub, we ensure that the generated formulas can effectively handle common edge cases and potential problems. Specific mutation rules are designed based on different logical types to ensure that the generated formulas strictly adhere to SMT syntax requirements. For example, for logical types that do not support strings, we specifically emphasize avoiding the use of string-related functions. These dynamically updated prompts help us continuously optimize the quality, accuracy, and adaptability of the generated formulas, ensuring they comply with specifications and meet high standards.

To further enhance the diversity and controllability of the mutation process, we designed a structured set of mutation rules for each logical type, which the LLM must strictly follow during the generation of each variant. These rules are categorized into four major levels:

- Syntax-Level: including variable/function renaming, whitespace and comment perturbation, and addition or removal of redundant declarations;
- Expression-Level: such as injecting boundary values (e.g., empty strings, hexadecimal patterns), mutating regular expression quantifiers, and introducing type conversion chains;
- Structure-Level: including addition or removal of assertions, reordering and splitting of expression blocks, and adjustment of variable bindings;
- Assertion-Level: such as injecting tautologies or contradictions, duplicating and modifying existing assertions, and pruning non-critical constraints.

To ensure the quality of the generated formulas, we have implemented a strict validation mechanism. To ensure compliance with the SMT-LIB standard, a validation check is performed on the generated SMT formulas. If syntax errors or logical inconsistencies are detected, the system returns detailed error messages to help the model self-correct. If the same file exceeds five errors, the system triggers a re-mutation or re-fetches the seed, accelerating the mutation process while ensuring the quality and accuracy of the generated formulas. Through this feedback mechanism, we effectively improve the stability and reliability of the generated formulas, ensuring the efficiency and effectiveness of the entire mutation process.

5) Population Maintenance: We set a time threshold, and when the runtime difference between versions exceeds this threshold, we consider it as a test case potentially causing performance regression and select the relevant cases. Subsequently, we supplement these test cases by introducing a new SMT2 file to ensure the stability of the population size.

C. Regression Case Localization

After successfully detecting a regression case, the next core challenge is quickly locating the commit that introduced the issue. To improve the efficiency of regression localization, we have developed a two-stage localization framework based on the SPRFinder [31]localization method, which strengthens the ability to perform batch regression localization. First, we use coverage analysis to filter out commits that are unrelated to the problem, thereby narrowing down the suspicious range. Then, we apply an enhanced binary search strategy for localization. This layered localization mechanism significantly reduces testing costs by filtering out invalid commits in the early stages, while the binary search improves localization accuracy, effectively addressing the efficiency bottleneck that traditional methods face in large-scale commit scenarios.

1) Filtering Unrelated Commits.: In the process of batch regression localization, the first step is to filter out commits that are unrelated to the issue. First, we determine the scope of changes in each commit based on the code differences between different versions. Then, during the execution of the previous regression cases, we use the git diff tool to identify code differences between versions and apply instrumentation to obtain the coverage of the diff code, thereby identifying the covered lines. Next, by analyzing the coverage of these lines,

we assess which changes are related to the regression issue. In most cases, code changes that are not covered by the test logic are unlikely to cause performance degradation, so they can be marked as low-risk and excluded from further testing. By filtering out a large number of irrelevant changes, we can significantly narrow down the range of commits that require in-depth analysis and testing, thus improving the efficiency of regression localization.

2) Performance Testing Binary Search Localization. : Once the range of version commits related to the regression issue has been determined, we apply a performance testing binary search method based on time thresholds to precisely locate the commit that triggered the performance change. First, assuming the current commit range is from the starting commit C_start to the ending commit C_end, we select the middle commit C_mid, dividing the range into two parts. For each test case t, we measure its execution time on the starting commit, middle commit, and ending commit. Since execution times may fluctuate, comparing time differences alone could lead to misinterpreting performance fluctuations as significant changes. To avoid misjudgment and maintain sensitivity, we set one-third of the time difference as the threshold, focusing on noticeable performance changes.Specifically, if $|t_{\text{mid}} - t_{\text{start}}| < \frac{|t_{\text{start}} - t_{\text{end}}|}{3}$, it indicates that the performance of the middle commit is closer to the starting commit, and the performance change is likely to occur in the latter half of the range. If $|t_{\text{mid}} - t_{\text{end}}| < \frac{|t_{\text{start}} - t_{\text{end}}|}{3}$, it suggests that the performance of the middle commit is closer to the ending commit, and the performance change is likely to occur in the first half of the range. The range is then gradually narrowed down, ultimately pinpointing the commit that caused the performance regression.

IV. EVALUATION OF SMTPRT

In this section, we evaluate our method to answer the following research questions:

RQ1: How effective is the SMTPRT method in generating regression error test cases across various logic types?

RQ2: How effective are key components in improving the generation of regression error cases?

RQ3: How effective is the SMTPRT method in improving the efficiency of regression localization?

A. Experimental Setups

1) Benchmark: We selected the **Z3 solver** [8](version 4.13.4) and the **CVC5 solver** [1](version 2.1.1) as the main experimental subjects. These versions represent continuous updates in functionality and performance, making it easier to assess the regression and performance changes between different versions. To ensure fairness, we used randomly generated SMT2 files from SPRFinder as the initial seeds, and set a 20-second timeout for each test case. Test cases exceeding this time limit were marked as failed.

2) Genetic Algorithm Setup: In the genetic algorithm, to emphasize the impact of time difference on regression error detection, we weighted the fitness function, setting ω_1=0.85 and ω_2=0.15. To improve the reliability of the experiment, we set 3 individuals to compete in tournament selection, ensuring selection pressure while maintaining appropriate population diversity. The mutation operation used the **Deepseek-r1** model with a temperature setting of 0.9, which is effective in identifying SMT logical types and semantics. To reduce the randomness of the experiment, we repeated the experiment 5 times.

3) Baselines: To demonstrate the validity of SMTPRT, we use the latest and most effective method, SPRFinder, as the baseline for two sets of comparison experiments: regression testing and fault localization. In the regression testing group, we also introduce a random mutation method, which generates test cases randomly for regression error detection. For the fault localization experiments, we use the traditional binary search-based tool, Git-bisect, as a comparison. Through these comparison experiments, we can comprehensively evaluate the advantages of SMTPRT in performance regression detection and fault localization.

B. RQ1: Validating the validity of the SMTPRT

1) Setup(RQ1): We ran the random mutation, SPRFinder, and SMTPRT methods on the target solvers Z3 and CVC5, with a testing duration of 24 hours, aiming to generate test cases that trigger regression errors. To ensure fairness and consistency, we used the random SMT2 files generated by SPRFinder as the initial seeds for executing all three methods. In the SMTPRT method, a large language model was used to generate 10 mutated SMT2 files for each test case. We compared the validity of different methods based on the number of test cases that triggered regression errors.

2) Results: Figure 2 and 3 show the results after 24 hours of experimentation with three different methods. As shown in Figure 2, when testing the Z3 solver, the random mutation, SPRFinder, and SMTPRT methods detected 0, 5, and 20 regression cases on average, respectively. Similarly, as shown in Figure 3, when testing the CVC5 solver, the three methods detected 0, 4, and 11 regression cases, respectively. Overall, within the same time frame, random mutation failed to generate valid regression test cases, while the SMTPRT method significantly outperformed both random mutation and SPRFinder, indicating that our method performs better in this task.

TABLE 2: Types of logic that can be tested

Methods	QF_S	QF_BV	QF_UF	QF_NRA	QF_FP	QF_NIA
StringFuzz [4]	✓	✗	✗	✗	✗	✗
BanditFuzz [19]	✓	✓	✗	✓	✓	✓
SPRFinder [31]	✓	✗	✗	✗	✗	✗
SMTPRT	✓	✓	✓	✓	✓	✓

In Table 2, we compare the SMT logic types covered by our method with several baseline methods. As shown in the table, StringFuzz can only test string-related interpreters (e.g., QF_S) and cannot effectively test other types of SMT logic. BanditFuzz can test a broader range of logic types, including QF_S, QF_BV, QF_FP, QF_NIA and QF_NRA, but

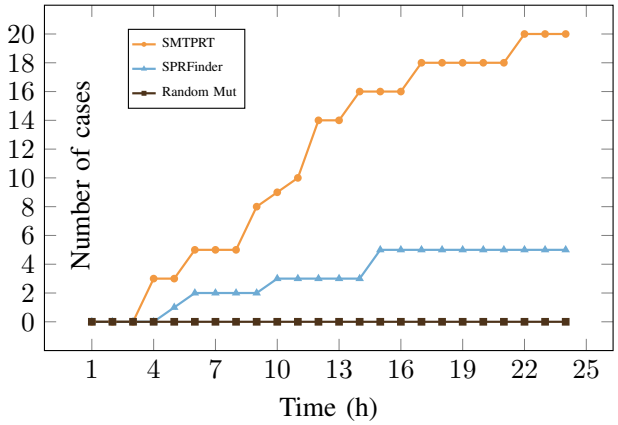


Fig. 2: Regression cases generated by Z3

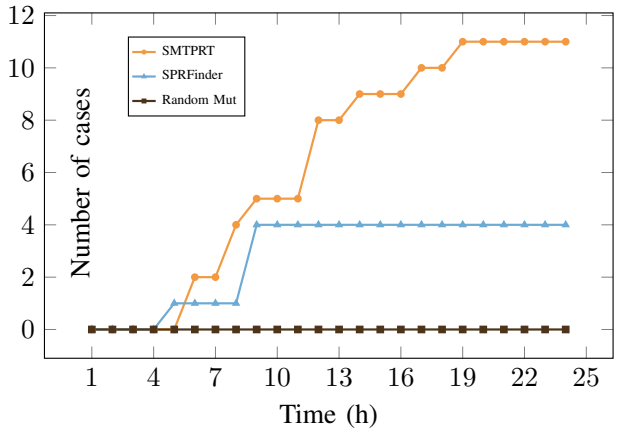


Fig. 3: Regression cases generated by CVC5

TABLE 3: The number of regression cases for different logic types in 24 hours

Solver	QF_S	QF_BV	QF_NIA	QF_UF	QF_NRA	QF_FP
Z3	20	5	3	3	2	3
CVC5	11	3	5	2	2	0

still cannot cover logic types such as QF_UF. SPRFinder, like StringFuzz, is limited to testing string-related interpreters and cannot be extended to other types of SMT logic. In contrast, the SMTPRT method, through large language model mutations, can easily test a wider range of SMT solver logic types and, by modifying the prompts, can flexibly adapt to different logic types.

To evaluate the performance of the SMTPRT method across various SMT logic types, we selected six logic types, which were based on commonly used single-query track types in SMT-COMP [21] and their frequency in real-world applications. These six logic types include QF_S, QF_BV, QF_NIA, QF_UF, QF_NRA and QF_FP. We customized the corresponding test prompts based on the official benchmark library and relevant literature. Table 3 shows the number of regression error test cases generated by the SMTPRT method for these six logic types within 24 hours.

Answer to RQ1: The SMTPRT method significantly outperformed random mutation and SPRFinder in generating regression error test cases. During the 24-hour test, SMTPRT successfully detected more regression error test cases and covered a wider range of SMT logic types, demonstrating stronger performance and broader applicability in regression detection tasks.

C. RQ2: Impact of Key Components on SMTPRT Performance

1) Setup(RQ2): To systematically evaluate the impact of key components, including the large language model, code coverage, and time score, on the performance of SMTPRT, we designed and implemented a series of experiments, each focusing on the detailed assessment of one component. For the large language model, we conducted an evaluation based on two dimensions: the number of regression error cases detected and the validity of the generated test cases, analyzing its performance and syntactic correctness in regression testing. For code coverage and time score, we designed a fitness function and assessed their role in regression detection through the number of regression cases identified.

First, we conducted comparative experiments across multiple mainstream large language models to assess their performance in generating SMT regression error cases. The evaluated models included CodeT5, LLaMA3.2, Qwen2.5-Coder-32B-Instruct, Mistral, Deepseek-v3, and Deepseek-r1. Due to potential semantic errors encountered by large language models when interpreting SMT logical expressions, the generated SMT files may contain syntax errors, logical inconsistencies, or become unrecognizable by solvers, which could ultimately affect the validity of the tests. Therefore, in these experiments, we not only focused on the number of detected regression cases generated within 12 hours but also paid particular attention to the validity and quality of the generated cases. To evaluate this, we employed an validity metric, which is defined as the proportion of valid .smt2 files, generated by the large language model when mutating the seed files, that adhere to SMT syntax requirements.

Based on the selection of the optimal model, we further designed ablation experiments with fitness functions in three different scenarios to analyze the impact of using time difference, coverage, and the combination of both as the fitness function for generating regression error cases. All experiments used different logic types, and the testing duration was fixed at 24 hours.

2) Results: As shown in Figure 4, the comparative experimental results indicate that CodeT5, LLaMA3.2, and Qwen2.5 performed poorly in generating valid regression cases. The primary reason for this may be their limited adaptability in generating descriptive text. These models, being relatively smaller in scale, lack sufficient contextual understanding and

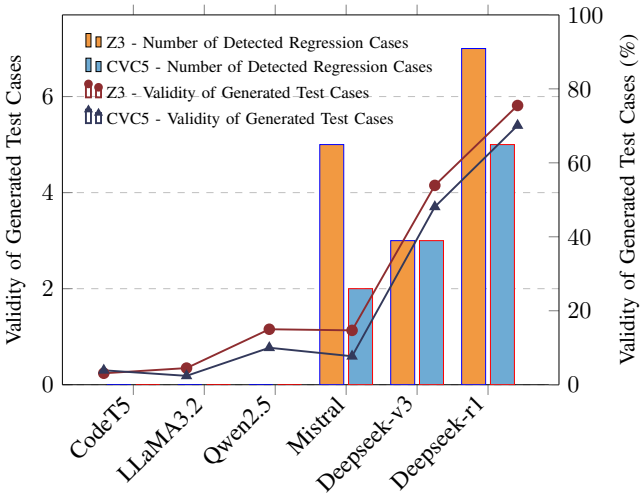


Fig. 4: Comparison of Number of Detected Regression Cases and Validity of Generated Test Cases Across Six LLMs (Z3 and CVC5)

TABLE 4: Fitness Function Comparison Experiment

	Time_score			Code_coverage			SMTPRT		
	min	max	avg	min	max	avg	min	max	avg
Z3	13	20	16	0	3	2	13	24	20
CVC5	5	10	8	0	3	2	9	13	11

syntactic reasoning capabilities, which hinder their ability to accurately generate valid SMT files that comply with SMT syntax during the mutation process. In contrast, Mistral, while generating SMT cases the fastest, demonstrated a low validity in its regression cases, resulting in many cases being unable to be correctly parsed by solvers. Deepseek-v3, although capable of generating regression cases, failed to achieve the level of validity and quantity reached by Deepseek-r1. Deepseek-r1, on the other hand, outperformed all models across all evaluation metrics, demonstrating its superior ability in generating regression cases. Therefore, Deepseek-r1 was selected as the large language model for generating regression test cases in the SMTPRT method.

As shown in Table 4, when only time difference was used as the selection criterion, Z3 generated 16 regression cases and CVC5 generated 8 regression cases; when only coverage was used, Z3 and CVC5 each generated only 2 regression cases. By combining time difference and coverage as the selection criterion, and using a weighted fitness function, a total of 31 regression cases were generated within the fixed time frame, resulting in an increase in the number of cases.

The results show that when relying solely on time difference or coverage, there were significant differences in both the quantity and quality of the generated test cases. However, when time difference and coverage were combined, the generated test cases became more balanced, avoiding duplicate regression error cases and more effectively covering potential performance regression issues. Introducing differential coverage significantly improved the diversity and coverage of the test cases, which is crucial for triggering different types of regression errors.

Answer to RQ2: By selecting Deepseek-r1 as the underlying large language model and combining time difference and code coverage into a weighted fitness function, the SMTPRT method achieved significant improvements in generating regression test cases. This enhancement increased the diversity and coverage of the test cases, making them more effective in triggering various types of regression errors. Therefore, this combination is a key component of the SMTPRT method.

D. RQ3: Regression Bug Localization

1) Setup(RQ3): To further validate the validity of the proposed commit-level fine-grained localization method, we conducted a comparative experiment between Git-bisect, SPRFinder, and SMTPRT. The experiment used total time and average time as evaluation metrics, providing an intuitive reflection of the efficiency differences between the three methods in actual localization tasks. To obtain coverage, we first use the GCC compiler to perform instrumentation compilation, generating executable files that contain coverage detection logic. Then, we use the gcov tool to generate coverage data for each version and utilize the git diff command to identify the code differences between versions, calculating the coverage of the modified code.

TABLE 5: Localization time of regression cases (hour)

Solver	Method	QF_S	QF_BV	QF_NIA	QF_UF	QF_NRA	QF_FP
Z3	Git-bisect	0.83	0.8	0.69	0.58	0.68	0.91
	SPRFinder	0.21	0.63	0.49	0.42	0.47	0.76
	SMTPRT	0.18	0.54	0.44	0.36	0.43	0.57
CVC5	Git-bisect	0.77	0.64	0.58	0.74	0.69	0
	SPRFinder	0.4	0.55	0.32	0.7	0.67	0
	SMTPRT	0.35	0.41	0.27	0.64	0.61	0

2) Results: Table 5 shows the average time required to successfully locate 59 regression cases, which were obtained from previous regression testing. Due to the small number of cases for some logic types, batch testing could not be effectively performed, which may lead to some issues with the results. Therefore, we chose to analyze the QF_S logic type in Z3, which had the largest number of generated cases. The average localization times for each regression case were 0.83 hour, 0.21 hour, and 0.18 hour, respectively. Clearly, the SMTPRT method significantly outperforms the traditional Git-bisect method in localization efficiency. Compared to SPRFinder, within the same time frame, SMTPRT is able to locate approximately 16.7% more regression cases. Additionally, during the experiment, we found that in the step of filtering irrelevant commits, we successfully removed approximately 39% of the commits.

Answer to RQ3: The SMTPRT method significantly improves the localization efficiency of regression errors by narrowing the coverage range of differential code and incorporating an enhanced binary search strategy. Compared to the current method, SMTPRT is able to locate approximately 16.7% more regression test cases within the same time frame and effectively filters out around 39% of irrelevant commits, significantly accelerating the analysis and debugging process.

V. Threats to Validity

First, the seed SMT test files selected from the official benchmark library used in the experiments may introduce some bias. To minimize this effect, we conducted multiple experiments and averaged the results.

Additionally, the 20-second timeout set during testing may not be optimal, which could result in some test cases failing to detect regression errors due to time constraints.

The use of a single performance regression time threshold could also impact the experimental results. Differences in the chosen threshold might lead to variations in the research conclusions, and it is essential to carefully determine evaluation standards in related studies.

In terms of reliability, differences in hardware and software environments may affect the experimental outcomes. To ensure the stability of the results, future experiments should be validated and repeated across multiple environments.

VI. Related Works

In recent years, SMT solver testing techniques have expanded from functional verification to performance optimization, forming a multidimensional testing system that balances both functionality and performance, promoting the efficient and reliable application of solvers in industrial scenarios.

Early research focused on the automated detection of basic functional defects. FuzzSMT [5], one of the early testing tools, generates SMT formulas randomly and uses fuzz testing to reveal solver defects through extensive random testing, achieving significant results in capturing solver crashes and invalid model errors.

As testing scenarios have become more complex, OpFuzz [23] and [17] generate type-correct mutated formulas by mutating operators that conform to type constraints. This significantly enhances the detection of functional errors, particularly correctness errors, invalid model errors, and crashes. Yao's team introduced the Sparrow tool and the innovative Skeletal Approximation Enumeration (SAE) method [26], which generates semantically equivalent variants through predicate symbol substitution and formula expansion. They constructed a metamorphic testing framework based on logical approximation relations, enabling the generation of diverse test cases.The SMT2Test [30] framework generates diversified test cases for SMT solver validation by utilizing seed SMT formulas, along with formula mutators and transformation components, when testing program verifiers. However, the framework has only been tested on three specific logical types: QF_LIA, QF_LRA, and QF_NRA, and has not yet been extended to test other logical types.

BanditFuzz [19] focuses on performance errors, aiming to reveal efficiency differences between solver implementations by identifying algorithm bottlenecks through differential input construction strategies. However, it does not test multiple versions of the same solver. To address this, SPRFinder [31] was introduced specifically for detecting performance regression in SMT solvers. SPRFinder builds upon BanditFuzz, focusing on performance regression in string solvers, and uses an adaptive test case generation strategy and a dynamic time warping (DTW) [11] distance reward mechanism. Unlike the Git bisect strategy proposed by Coulder et al. [7], SPRFinder processes test cases in batches, enabling rapid identification of the root cause of defects.

The SMTPRT method proposed in this paper builds on SPRFinder and introduces an innovative improvement. Dominated by a genetic algorithm, it generates various SMT2 test cases covering multiple logical types through large language models. SMTPRT offers a more intelligent generation strategy aimed at triggering performance regression issues. Furthermore, by incorporating version difference analysis and code instrumentation techniques for commit filtering, SMTPRT provides a faster localization method for performance regression detection, enhancing both the accuracy and efficiency of the detection process. Compared to the existing method SPRFinder, our work can cover multiple types of SMT theory and enables faster localization.

VII. Conclusion

This paper introduces a method called SMTPRT, which effectively addresses the performance regression detection and localization issues in SMT solvers. SMTPRT generates test cases through a genetic algorithm, utilizing large language model mutations combined with time difference and coverage strategies for selection. For locating regression errors, differential code coverage analysis and enhanced binary search are used, significantly improving detection and localization efficiency. Experimental results show that, compared to traditional methods, SMTPRT is more effective in performance regression testing for SMT interpreters, providing greater logical coverage and demonstrating its advantages.

Despite the significant results achieved by SMTPRT, there is still room for optimization. The current experiments were conducted only on Z3 and CVC5 solvers, and future work should extend to other commonly used solvers to evaluate its generalizability. Additionally, the current localization granularity is at the commit level, and future work could improve this to function-level or line-level localization, enhancing both efficiency and accuracy. Future research should focus on more granular localization techniques to help developers quickly fix performance regression issues.

ACKNOWLEDGMENT

This work was supported by the National Key Research and Development Program of China (2023YFB3107100), and the National Natural Science Foundation of China (Key Program, Grant No. 62332005). The opinions, findings, and conclusions expressed in this paper are solely those of the authors and do not reflect the views of the funding agencies.

REFERENCES

[1] Haniel Barbosa, Clark Barrett, Martin Brain, Gereon Kremer, Hanna Lachnitt, Makai Mann, Abdalrhman Mohamed, Mudathir Mohamed, Aina Niemetz, Andres Nötzli, Alex Ozdemir, Mathias Preiner, Andrew Reynolds, Ying Sheng, Cesare Tinelli, and Yoni Zohar. cvc5: A versatile and industrial-strength smt solver. In Dana Fisman and Grigore Rosu, editors, *Tools and Algorithms for the Construction and Analysis of Systems*, pages 415–442, Cham, 2022. Springer International Publishing.

[2] Clark Barrett, Aaron Stump, Cesare Tinelli, Sascha Boehme, David Cok, David Deharbe, Bruno Dutertre, Pascal Fontaine, Vijay Ganesh, and Alberto Griggio. C.: The smt-lib standard: Version 2.0. *Jp J.geom.topol*, 53(3):273–284, 2010.

[3] Nikolaj Bjørner and Leonardo de Moura. Applications of smt solvers to program verification. *Notes for the Summer School on Formal Techniques*, 2014.

[4] Dmitry Blotsky, Federico Mora, Murphy Berzish, Yunhui Zheng, Ifaz Kabir, and Vijay Ganesh. Stringfuzz: A fuzzer for string solvers. In Hana Chockler and Georg Weissenbacher, editors, *Computer Aided Verification*, pages 45–51, Cham, 2018. Springer International Publishing.

[5] Robert Brummayer and Armin Biere. Fuzzing and delta-debugging smt solvers. In *Proceedings of the 7th International Workshop on Satisfiability Modulo Theories*, SMT '09, page 1–5, New York, NY, USA, 2009. Association for Computing Machinery.

[6] Alessandro Cimatti, Alberto Griggio, Bastiaan Joost Schaafsma, and Roberto Sebastiani. The mathsat5 smt solver. In Nir Piterman and Scott A. Smolka, editors, *Tools and Algorithms for the Construction and Analysis of Systems*, pages 93–107, Berlin, Heidelberg, 2013. Springer Berlin Heidelberg.

[7] C. Couder. Fighting regressions with git bisect. https://www.kernel.org/pub/software/scm/git/docs/git-bisect-lk2009.html. The Linux Kernel Archives, GIT Repository, Version, vol. 4, no. 5, 2008.

[8] Leonardo de Moura and Nikolaj Bjørner. Z3: An efficient smt solver. In C. R. Ramakrishnan and Jakob Rehof, editors, *Tools and Algorithms for the Construction and Analysis of Systems*, pages 337–340, Berlin, Heidelberg, 2008. Springer Berlin Heidelberg.

[9] Luciana Brasil Rebelo dos Santos, Erica Ferreira de Souza, Andre Takeshi Endo, Catia Trubiani, Riccardo Pinciroli, and Nandamudi Lankalapalli Vijaykumar. Performance regression testing initiatives: a systematic mapping. *Information and Software Technology*, 179:107641, 2025.

[10] Bruno Dutertre. Yices2.2. In Armin Biere and Roderick Bloem, editors, *Computer Aided Verification*, pages 737–744, Cham, 2014. Springer International Publishing.

[11] Toni Giorgino. Computing and visualizing dynamic time warping alignments in r: The dtw package. *Journal of Statistical Software*, 31(7):1–24, 2009.

[12] Jongwook Kim, Sunbeom So, and Hakjoo Oh. Diver: Oracle-guided smt solver testing with unrestricted random mutations. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, pages 2224–2236. IEEE, 2023.

[13] Sebastian Kühne. Automatically explaining and localizing semantic errors in smt solvers. 2024.

[14] Ai Liu, Yang Liu, and Shaoying Liu. Tbfv4j: An automated testing-based formal verification tool for java. In *Proceedings of the 34th ACM SIGSOFT International Symposium on Software Testing and Analysis*, pages 66–70, 2025.

[15] Muhammad Numair Mansur, Maria Christakis, Valentin Wüstholz, and Fuyuan Zhang. Detecting critical bugs in smt solvers using blackbox mutational fuzzing. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ESEC/FSE 2020, page 701–712, New York, NY, USA, 2020. Association for Computing Machinery.

[16] Aina Niemetz and Mathias Preiner. Bitwuzla. In Constantin Enea and Akash Lal, editors, *Computer Aided Verification*, pages 3–17, Cham, 2023. Springer Nature Switzerland.

[17] Jiwon Park, Dominik Winterer, Chengyu Zhang, and Zhendong Su. Generative type-aware mutation for testing smt solvers. *Proceedings of the ACM on Programming Languages*, 5(OOPSLA):1–19, 2021.

[18] Sebastian Pointner, Sven Wenzek, and Robert Wille. Smt-based placement for system-on-chip design. pages 1–5, 05 2021.

[19] Joseph Scott, Federico Mora, and Vijay Ganesh. Banditfuzz: A reinforcement-learning based performance fuzzer for smt solvers. In *Software Verification: 12th International Conference, VSTTE 2020, and 13th International Workshop, NSV 2020, Los Angeles, CA, USA, July 20–21, 2020, Revised Selected Papers*, page 68–86, Berlin, Heidelberg, 2020. Springer-Verlag.

[20] Jingyi Shi, Yang Xiao, Yuekang Li, Yeting Li, Dongsong Yu, Chendong Yu, Hui Su, Yufeng Chen, and Wei Huo. Acetest: Automated constraint extraction for testing deep learning operators. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*, pages 690–702, 2023.

[21] SMT-COMP. The international smt competition. https://smt-comp.github.io/2024/, 2024. Retrieved 2024-10-16.

[22] Xuezhi Song, Yun Lin, Siang Hwee Ng, Yijian Wu, Xin Peng, Jin Song Dong, and Hong Mei. Regminer: towards constructing a large regression dataset from code evolution history. In *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis*, pages 314–326, 2022.

[23] Dominik Winterer, Chengyu Zhang, and Zhendong Su. On the unusual effectiveness of type-aware operator mutations for testing smt solvers. *Proc. ACM Program. Lang.*, 4(OOPSLA), November 2020.

[24] Dominik Winterer, Chengyu Zhang, and Zhendong Su. Validating smt solvers via semantic fusion. In *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI 2020, page 718–730, New York, NY, USA, 2020. Association for Computing Machinery.

[25] Ziyi Yang, Guangyu Hu, Mingkai Miao, Changyuan Yu, and Hongce Zhang. Smt-sweep: Word-level representation unification for hardware verification. *arXiv preprint arXiv:2507.02008*, 2025.

[26] Peisen Yao, Heqing Huang, Wensheng Tang, Qingkai Shi, Rongxin Wu, and Charles Zhang. Skeletal approximation enumeration for smt solver testing. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ESEC/FSE 2021, page 1141–1153, New York, NY, USA, 2021. Association for Computing Machinery.

[27] Fangke Ye, Jisheng Zhao, Jun Shirako, and Vivek Sarkar. Concrete type inference for code optimization using machine learning with smt solving. *Proc. ACM Program. Lang.*, 7(OOPSLA2), oct 2023.

[28] Leyi Yin, Yue Deng, and Peng Li. Verifying dynamic properties of nonlinear mixed-signal circuits via efficient smt-based techniques. In *Proceedings of the International Conference on Computer-Aided Design*, ICCAD '12, page 436–442, New York, NY, USA, 2012. Association for Computing Machinery.

[29] Boyan Yordanov, Christoph M. Wintersteiger, Youssef Hamadi, and Hillel Kugler. Smt-based analysis of biological computation. In Guillaume Brat, Neha Rungta, and Arnaud Venet, editors, *NASA Formal Methods*, pages 78–92, Berlin, Heidelberg, 2013. Springer Berlin Heidelberg.

[30] Chengyu Zhang and Zhendong Su. Smt2test: From smt formulas to effective test cases. *Proc. ACM Program. Lang.*, 8(OOPSLA2), October 2024.

[31] Yao Zhang, Xiaofei Xie, Yi Li, Yun Lin, Sen Chen, Yang Liu, and Xiaohong Li. Demystifying performance regressions in string solvers. *IEEE Transactions on Software Engineering*, 49(3):947–961, 2023.